/dev/input/event0。

7.4.9 中断传输：小、周期、低延迟

上一节把一份报告到手之后的软件分发讲完了，这一节回到总线上，看一次 interrupt 传输到底搬了些什么。§7.4 开头已经点破“interrupt”这个名字的误会：设备从不主动打断主机，是主机每 bInterval 毫秒替 CPU 问一次。interrupt 与 bulk 用的是同一套可靠机制（Handshake 加 DATA0/DATA1 翻转），唯一的区别就是这条周期承诺——主机必须按时来问，于是延迟有界；代价是每次问的量很小（不超过端点的 mps），正合 HID 按键这种“来得快、内容短”的负载。

把这三句话落到一次真实按键上。按下字母 l（没有配合任何修饰键）时，串口上会滚出三行：

```
dwc2: ch6 transfer complete addr=4 ep=1 bytes=8 data=0,0,f,0,0,0,0,0
usbkbd: channel 6 IRQ received
usbkbd: first HID report mod=0 keys=f,0,0,0,0,0
```

这三行正好是三层各说一句。第一行来自 HCD（dwc2.c）：ch6 这一轮 interrupt-IN 完成，HCINT 的传输完成位置位，DMA 已经把 bytes=8 个字节写进内存，于是把缓冲区原样打出来——addr=4 ep=1 说明这是对地址 4 的设备、端点 1（IN）的轮询。第二行是 USB core 经唯一入口 usb_hcd_giveback_urb() 回调进类驱动（边界 2，见 §7.4“内核软件框架”一节），usbkbd.c 报告“ch6 的事件到了”。第三行才是类驱动把这 8 字节拆开读：mod=0 是修饰位图，keys=f,... 是六个键槽的内容。

8 字节报文的逐字段格式。 Boot Keyboard（HID boot protocol）的报告是定长 8 字节，一次 interrupt-IN 恰好搬一个完整报告，所以 bytes=8 就等于 mps，不需要像 bulk 那样靠“短包”标志结束：

表 7.8: 按下 l 时那 8 字节报告（0,0,f,0,0,0,0,0）的逐字段拆解

偏移	字段	本次值	含义
byte 0	Modifier 位图	0x00	八个修饰键各占一位（bit0–3 = 左 Ctrl/Shift/Alt/GUI，bit4–7 = 右侧四个），全 0 表示没按任何修饰键
byte 1	Reserved	0x00	Boot 协议固定保留字节，恒 0
byte 2	Key 0	0x0F	第一个键槽：HID usage 0x0f，即字母 l
byte 3–7	Key 1–5	0x00	其余五个键槽，0x00 = 该槽无按键（一份报告最多同时报 6 个键）

关键在于 byte 2 的 0x0f 是 *HID usage ID*，不是 ASCII。HID 键盘用途表从 0x04 起排字母，0x04 = a，所以 l 是第 12 个字母、usage = 0x04+11 = 0x0f；而 l 的 ASCII 是 0x6c，两者毫无关系。这正是 usbkbd.c 里 keymap[]/hid_to_key[] 要按 *usage* 编号索引（[4]='a'）而不能直接当字符输出的原因。六个键槽全 0x00 才表示“没键按下”，而 0x01 ~ 0x03 是保留的错误码（rollover 等），驱动会原样保留上一份报告——这条特判在 §7.4.8 展开。

总线上的一来一回。 这 8 字节是被一次普通的 IN 事务取回的，三相结构和 §7.4 里画的完全一样，只是没有 SETUP/STATUS 那套“先谈请求”的外壳：

```
主机每 bInterval(10 ms) 用 ch6 发起一次 IN:
  IN token    ADDR=4 ENDP=1              主机 -> 键盘   "EP1 有报告吗？"
键盘有新数据时:
  DATA0/DATA1  [8 字节负载] CRC16         键盘 -> 主机   <- 就是 data=0,0,f,0,0,0,0,0
  ACK                                    主机 -> 键盘   "收下"
键盘没有键按下时:
  NAK                                    键盘 -> 主机   "暂无数据", 本轮不算完成, ch6 稍后重新 arm
```